

PROJECT REPORT

Handwritten Digit Recognition

Using Convolutional Neural Networks on the MNIST Dataset

Submitted By	Shanzay Atta-BSE233043 Maryam Fraz-BSE233031 Ayesha Khalid-BSE233046
Course	Artificial Intelligence
Instructor	Sir Temour Abbas
Institution	CUST
Submission Date	09-06-2026

Abstract

This report presents the design and implementation of a handwritten digit recognition system built using a Convolutional Neural Network (CNN). The system was trained on the MNIST dataset, which contains 70,000 grayscale images of handwritten digits from 0 to 9. The project covers the complete pipeline from data preprocessing and model design to training, evaluation, and deployment as a working desktop application. The trained model achieved a test accuracy of over 98% on the standard MNIST test set. An interactive application was also developed using Python's Tkinter library, allowing users to draw a digit on screen and receive a real-time prediction. This project helped in understanding the practical aspects of building and deploying a deep learning model, including the challenges that come with real-world input data that differs from the training distribution.

1. Introduction

Pattern recognition has been one of the core problems in computer science and artificial intelligence for a long time. Among its many sub-problems, handwritten digit recognition stands out as a classic and well-understood task that still has real practical value. Banks process thousands of handwritten cheques every day. Postal services need to read zip codes written by hand. Government forms, medical records, and educational assessments often involve handwritten numbers that need to be digitized. All of these applications rely on some form of digit recognition.

Before deep learning became popular, digit recognition systems used hand-crafted features and classical machine learning methods like Support Vector Machines or k-Nearest Neighbors. These methods required significant manual effort to define the right features and still struggled with the natural variation in human handwriting.

Convolutional Neural Networks changed this. Instead of manually defining features, CNNs learn them directly from raw pixel data through a series of convolutional and pooling layers. This approach is much better suited to image data and has consistently outperformed older methods on benchmarks like MNIST.

This project was built as a semester-long exercise to go through the full process of creating such a system — from loading and preprocessing data, to building and training a CNN, evaluating its performance, and finally deploying it in a simple interactive application. The goal was not just to hit a high accuracy number but to understand each step and what affects the outcome.

2. Problem Statement

The problem this project addresses is image classification — specifically, classifying a 28x28 pixel grayscale image of a handwritten digit into one of ten categories (digits 0 through 9). The input is a single image, and the expected output is a digit label along with a confidence score.

While this might sound straightforward, there are real complications. People write digits differently. A '1' drawn by one person might look like a '7' to someone else. A '4' written quickly can look like a '9'. The model needs to handle this variation without being given any explicit rules about what each digit looks like — it has to figure that out from the training data.

There is also a secondary challenge that became apparent during this project: the model performs well on MNIST test images, but when the same model is used to predict digits drawn by hand on a computer canvas, the accuracy drops noticeably. This is because the style, stroke thickness, and centering of drawn digits can differ quite a bit from the cleaned-up images in MNIST. Addressing this gap between training data and real-world input was one of the main practical challenges of the project.

3. Objectives

The main objectives of this project were:

- **1.** Understand the structure of the MNIST dataset and perform basic exploratory analysis.
- **2.** Build a data preprocessing pipeline that prepares images for model training.
- **3.** Implement a baseline dense neural network and measure its performance as a reference point.
- **4.** Design and train a CNN model and compare it against the baseline.
- **5.** Apply techniques like dropout, batch normalization, and data augmentation to improve generalization.
- **6.** Evaluate the trained model using accuracy, confusion matrix, and per-class metrics.
- **7.** Build an interactive desktop application where users can draw digits and get live predictions.
- **8.** Document the full process including what worked, what did not, and what could be improved.

4. Methodology and Working Process

4.1 Dataset

The MNIST dataset was used for training and evaluation. It contains 60,000 training images and 10,000 test images, each being a 28x28 grayscale image of a handwritten digit. The dataset is well-balanced across all ten classes and is available directly through the Keras API, so no manual downloading or file management was needed.

4.2 Data Preprocessing

Before feeding the data into the model, several preprocessing steps were applied. Pixel values were normalized from the range [0, 255] to [0, 1] by dividing by 255. This helps the model converge faster during training. The images were also reshaped from (28, 28) to (28, 28, 1) to add a channel dimension, which is required by convolutional layers in Keras. Labels were converted to one-hot encoded vectors for use with categorical cross-entropy loss.

Data augmentation was applied using Keras's ImageDataGenerator. Small random rotations, width and height shifts, zoom, and shear transformations were applied to the training images. This artificially increases the variety in the training data and helps the model become more robust to the natural variation in handwriting styles. Without augmentation, the model tends to overfit on the clean MNIST images and struggles with input that looks slightly different.

4.3 Model Architecture

Two models were built and compared. The first was a simple fully connected (dense) neural network, used as a baseline. It flattens the input image into a 784-dimensional vector and passes it through two dense layers. This model achieved around 97-98% accuracy but is limited in its ability to capture spatial patterns in the image.

The main model is a Convolutional Neural Network. It uses three convolutional blocks, each consisting of a Conv2D layer followed by batch normalization. MaxPooling layers are used after the first two blocks to reduce the spatial dimensions. Dropout layers were added after the pooling steps and before the final dense layer to reduce overfitting. The output layer uses softmax activation with 10 units, one for each digit class.

Layer	Type / Configuration	Purpose
1	Conv2D — 32 filters, 3x3, ReLU + BatchNorm	Extract low-level features like edges
2	Conv2D — 32 filters, 3x3, ReLU	Reinforce initial features
3	MaxPooling2D (2x2) + Dropout (0.25)	Reduce dimensions, prevent overfitting
4	Conv2D — 64 filters, 3x3, ReLU + BatchNorm	Learn more complex patterns
5	Conv2D — 64 filters, 3x3, ReLU	Deepen feature representation
6	MaxPooling2D (2x2) + Dropout (0.25)	Further reduction and regularization

Layer	Type / Configuration	Purpose
7	Flatten + Dense (256, ReLU) + BatchNorm	Fully connected classification layer
8	Dropout (0.5) + Dense (10, Softmax)	Final prediction with regularization

4.4 Training

The model was trained using the Adam optimizer with a learning rate of 0.001 and categorical cross-entropy as the loss function. A batch size of 64 was used, and training was run for up to 30 epochs. Early stopping was applied with a patience of 5 epochs, meaning training stopped automatically when the validation loss did not improve for 5 consecutive epochs. Ten percent of the training data was held out as a validation set to monitor overfitting during training.

Training was done on Google Colab using the free T4 GPU runtime. On CPU, training would take considerably longer, so the GPU was essential for completing the training in a reasonable time. The saved model was then downloaded and used locally in the desktop application.

4.5 Evaluation

After training, the model was evaluated on the 10,000 test images. The primary metric used was overall accuracy. A confusion matrix was also generated to see which digit pairs were most commonly confused. Per-class precision, recall, and F1-score were computed using Scikit-learn's classification report. Training and validation accuracy and loss were plotted over epochs to check for overfitting.

4.6 Interactive Application

A desktop application was built using Python's Tkinter library. The application displays a dark canvas where the user can draw a digit using the mouse. When the predict button is clicked, the drawn image is preprocessed — resized to 28x28 pixels, converted to grayscale, and normalized — and passed to the trained model. The predicted digit is shown along with its confidence score and a bar chart displaying the probability assigned to each of the ten digit classes.

5. Tools and Technologies Used

Tool / Library	Purpose
Python 3.12	Main programming language used throughout the project
TensorFlow / Keras	Building, training, and saving the CNN model
NumPy	Array operations and numerical data handling
Matplotlib	Plotting training curves and sample images
Seaborn	Visualizing the confusion matrix as a heatmap
Scikit-learn	Generating the classification report and confusion matrix
Pillow (PIL)	Image preprocessing for the Tkinter application input
Tkinter	Building the interactive digit drawing and prediction GUI
Google Colab	Cloud-based GPU environment used for training
GitHub	Version control and code storage

6. System Features and Modules

6.1 Data Pipeline Module

This module handles loading the MNIST dataset, normalizing pixel values, reshaping arrays, and applying data augmentation. It is the first stage in the pipeline and its output is fed directly into the training process.

6.2 Model Module

This module defines the CNN architecture and compiles it with the appropriate optimizer and loss function. It includes both the baseline dense model and the main CNN, making it easy to switch between them for comparison.

6.3 Training Module

The training module handles the `model.fit()` call with all callbacks configured. It saves the best version of the model based on validation loss and exports it as a `.keras` file for later use.

6.4 Evaluation Module

After training, this module loads the test set, generates predictions, and produces the accuracy score, confusion matrix, and classification report. All plots are saved as image files.

6.5 Interactive Application

The desktop application is a standalone Python script. It loads the saved model on startup and provides a drawing canvas. Key features include freehand drawing with the mouse, real-time prediction on button click, a confidence score display, a probability bar chart for all ten classes, and a clear button to reset the canvas.

7. Challenges Faced

7.1 Model Not Recognizing Hand-Drawn Digits Correctly

This was the most frustrating part of the project. The model would score over 98% on the MNIST test set but then misclassify clearly drawn digits in the application. A 7 would be predicted as a 1, or a 2 would come out as a 4. The reason is that MNIST digits are small, centered, and relatively consistent, while digits drawn on a large canvas by mouse tend to be thicker, positioned differently, and stylistically different. Retraining with stronger augmentation helped reduce this gap, but it did not completely eliminate it.

7.2 Installing TensorFlow on Windows

Installing TensorFlow on a Windows machine caused issues due to Windows Long Path restrictions. File paths inside the TensorFlow package are long enough to exceed the default Windows path length limit, causing the installation to fail. This was resolved by enabling long path support through the Windows Registry.

7.3 Tkinter Not Working on Google Colab

Tkinter requires a local display to render a window, which is not available on Colab since it runs on a remote server. The solution was to train the model on Colab, download the saved model file, and then run the Tkinter application locally on the development machine.

7.4 Confusion Between Visually Similar Digits

Certain digit pairs were consistently confused by the model — particularly 4 and 9, 3 and 8, and 1 and 7. These pairs share structural features that make them difficult to distinguish, especially when drawn casually. Increasing the model depth and applying stronger augmentation improved

performance on these pairs, but some confusion remained, which is expected given the nature of the problem.

8. Results and Output

8.1 Model Accuracy

The final CNN model achieved a test accuracy of approximately 98.5% to 99% on the MNIST test set, depending on the training run. This is a significant improvement over the baseline dense model, which typically scored around 97 to 97.5%. The improvement demonstrates the advantage of using convolutional layers for spatial feature extraction in image data.

8.2 Training Curves

The training and validation accuracy curves showed consistent improvement across epochs with no significant signs of overfitting. The gap between training and validation accuracy remained small throughout, which suggests the regularization techniques (dropout and batch normalization) were effective. Early stopping triggered before the maximum epoch count in most runs.

8.3 Confusion Matrix

The confusion matrix showed that the model performed well on most digit classes. The highest confusion rates were between digits 4 and 9, and between 3 and 8. These are also the pairs that humans sometimes confuse, so the model's behavior is consistent with the difficulty of the problem.

8.4 Interactive Application

The Tkinter application worked as intended for digits drawn clearly and centered on the canvas. The confidence bar chart was particularly useful — for correctly predicted digits, one bar was clearly dominant, while for incorrect predictions, the confidence was often split between two similar classes. This gave a good visual indication of where the model was uncertain.

9. Conclusion

This project successfully implemented a handwritten digit recognition system using a CNN trained on the MNIST dataset. The full pipeline was completed — from data loading and preprocessing, to model training and evaluation, to deployment in an interactive application. The final model performs well on standard test data and reasonably well on user-drawn input, with some limitations that are expected given the difference between the training distribution and real-world drawings.

The project was a useful hands-on introduction to deep learning. Going through each step manually made it much clearer why certain design choices matter — why convolutional layers are better than dense layers for images, why augmentation is necessary for generalization, and why a high benchmark accuracy does not always translate directly to strong real-world performance.

Overall, the project met its objectives. The model was trained, evaluated, and deployed, and the results were documented along with the challenges encountered along the way.

10. Future Improvements

There are several ways this project could be extended or improved:

- **Custom Training Data:** Collect a small dataset of user-drawn digit images and fine-tune the model on this data. This would significantly improve the application's accuracy on hand-drawn input.
- **Web Deployment:** Replace the Tkinter application with a web-based interface using Gradio or Flask, making it accessible from a browser without any local installation.
- **Multi-Digit Recognition:** Extend the system to recognize full multi-digit numbers instead of single isolated digits.
- **Character Recognition:** Apply the same CNN approach to letters and alphanumeric characters, moving from digit recognition to general OCR.
- **Better Input Preprocessing:** Add preprocessing steps to center and normalize the user's drawing before prediction, which would reduce the distribution gap between drawn digits and MNIST images.

11. References

LeCun, Y., Cortes, C., & Burges, C. J. C. (1998). The MNIST database of handwritten digits. Retrieved from <http://yann.lecun.com/exdb/mnist>

LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. Available at <https://www.deeplearningbook.org>

Chollet, F. (2021). *Deep Learning with Python* (2nd ed.). Manning Publications.

TensorFlow / Keras Documentation. (n.d.). Retrieved from https://www.tensorflow.org/api_docs

Scikit-learn Developers. (n.d.). `sklearn.metrics.classification_report`. Retrieved from https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_report.html

Simard, P. Y., Steinkraus, D., & Platt, J. C. (2003). Best practices for convolutional neural networks applied to visual document analysis. *Proceedings of ICDAR 2003*.